



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA
Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO
“POO matriz, áreas y perímetros”

Curso: Sistemas de Información

Docente: Ing. Breno Luque Zuñiga

Peralta Sebán, Daniel Sebastián (2024016803)

Tacna – Perú
2026



INDICE

I. INFORMACIÓN GENERAL	3
- Objetivos	3
- Equipos, materiales, programas y recursos utilizados	3
II. MARCO TEORICO	3
III. PROCEDIMIENTO	4
Programa: Multiplicación de Matrices:.....	4
1. Diseño del Programa.....	4
2. Tabla de Controles.....	5
3. Código del Programa.....	6
Programa: Áreas y perímetros:.....	12
1. Diseño del Programa.....	12
2. Tabla de Controles.....	13
3. Código del Programa.....	13
IV. ANALISIS E INTERPRETACION DE RESULTADOS	22
CONCLUSIONES	22
RECOMENDACIONES	23
WEBGRAFIA	23



INFORME DE LABORATORIO 1

TEMA: POO matriz, áreas y perímetros

I. INFORMACIÓN GENERAL

- **Objetivos:**
 - Aplicar principios de la Programación Orientada a Objetos para el lenguaje C#.
 - Comprender el uso de clases, objetos, constructores y métodos dentro de las aplicaciones Windows Forms.
 - Implementar operaciones matemáticas con matrices bidimensionales.
 - Desarrollar un sistema que permita hacer multiplicación de matrices mediante encapsulamiento y métodos estáticos.
 - Aplicar herencia y polimorfismo en el cálculo de áreas y perímetros de figuras geométricas.
 - Aplicar el diseño de interfaces gráficas Windows Forms de Visual Studio.
 - Validar la información introducida por el usuario usando estructuras condicionales y excepción.
- **Equipos, materiales, programas y recursos utilizados:**
 - Computadora con sistema operativo Windows 8
 - Visual Studio
 - Lenguaje de programación C#.
 - .NET Framework

II. MARCO TEORICO

Programación Orientada a Objetos (POO): La Programación Orientada a Objetos es un paradigma de programación para la creación de objetos que contienen atributos y métodos. Este paradigma permite organizar el software modular, reutilizable y fácil de mantener.

Los principios más importantes de la POO son:

Encapsulamiento: Proteger los datos internos de una clase usando atributos de tipo privado, y poder acceder a ellos a través de métodos o propiedades.

Herencia: Permitir que una clase hija pueda recibir las características y el comportamiento de una clase padre.

Polimorfismo: Permitir que las diferentes clases puedan implementar métodos de forma diferente usando una misma estructura.



Clases y Objetos: Una clase es una plantilla para crear objetos. Los objetos son entidades que tienen características y comportamientos definidos.

Windows Forms: Windows Forms es la biblioteca gráfica de .NET que se utiliza para crear aplicaciones gráficas de escritorio, que son las que tienen una interfaz visual, utilizando formularios, botones, cuadros de texto y tablas.

Matrices en C#: Una matriz bidimensional permite almacenar datos en filas y columnas.

Figuras Geométricas: Las figuras geométricas son representaciones matemáticas traducidas a los ordenadores y que permiten el cálculo de propiedades asociadas como el área o el perímetro.

III. PROCEDIMIENTO

Programa: Multiplicación de Matrices

1. Diseño del Programa

Multiplicación de Matrices

Filas A: Filas B:

Columnas A: Columnas B:

Matriz A Matriz B Matriz Resultado

* =



2. Tabla de Controles

Control	Propiedad	Valor
Label1	Text	Multiplicación de Matrices
	Font	bold type
	ForeColor	Highlight
Label2	Text	Filas A:
Label3	Text	Columnas A:
Label4	Text	Filas B:
Label5	Text	Columnas B:
Label6	Text	Matriz A
Label7	Text	Matriz B
Label8	Text	Matriz Resultado
Label9	Text	*
	Font	bold type
Label10	Text	=
	Font	bold type
Button1	Name	btnCrear
	Text	Crear Matriz
Button2	Name	btnMultiplicar
	Text	Multiplicar
Button3	Name	btnBorrar
	Text	Borrar
textBox1	Name	txtFilasA
textBox2	Name	txtFilasB
textBox3	Name	txtColumnasA
textBox4	Name	txtColumnasB
dataGridView1	Name	dgvA
dataGridView2	Name	dgvB
dataGridView3	Name	dgvR



3. Código del Programa

El proyecto está dividido en:

- Clase Matriz
- Formulario Form1

1. Clase Matriz

Declaración de la Clase

Aquí se define la clase llamada Matriz.

Encapsulamiento de atributos

Estos atributos almacenan:

- Filas: cantidad de filas.
- Columnas: cantidad de columnas.
- Datos: matriz bidimensional donde se guardan los números.

Se utiliza private para aplicar encapsulamiento y evitar modificaciones externas directas.

```
12 referencias
public class Matriz
{
    // Encapsulamiento
    private int filas;
    private int columnas;
    private int[,] datos;
```

Constructor

El constructor se ejecuta automáticamente cuando se crea un objeto de tipo Matriz.

```
// Constructor
3 referencias
public Matriz(int f, int c)
{
    filas = f;
    columnas = c;
    datos = new int[filas, columnas];
}
```

Propiedades

Permiten obtener los valores de filas y columnas sin acceder directamente a los atributos privados.

Propiedad Filas

```
// Propiedades
8 referencias
public int Filas
{
    get { return filas; }
}
```

Propiedad Columnas

```
9 referencias
public int Columnas
{
    get { return columnas; }
}
```

Método SetValor

Asigna un valor específico dentro de la matriz.

```
// Método para asignar valores
3 referencias
public void SetValor(int fila, int columna, int valor)
{
    datos[fila, columna] = valor;
}
```

Método GetValor

Obtiene un valor almacenado en una posición específica.

```
// Método para obtener valores
3 referencias
public int GetValor(int fila, int columna)
{
    return datos[fila, columna];
}
```

Método Multiplicar

Realiza la multiplicación entre dos matrices.

Se utiliza static porque el método pertenece a la clase y no a un objeto específico. Verifica si las matrices pueden multiplicarse.

```
// Método multiplicar matrices
1 referencia
public static Matriz Multiplicar(Matriz A, Matriz B)
{
    if (A.Columnas != B.Filas)
    {
        return null;
    }
}
```

Creación de matriz resultado

Crea una nueva matriz donde se almacenará el resultado final.

```
Matriz resultado = new Matriz(A.Filas, B.Columnas);
```

Estructuras repetitivas

Recorre las filas.

```
for (int i = 0; i < A.Filas; i++)
{
```

Recorre las columnas.

```
for (int j = 0; j < B.Columnas; j++)
{
    int suma = 0;
```

Realiza la multiplicación matemática.

```
for (int k = 0; k < A.Columnas; k++)
{
    suma += A.GetValor(i, k) * B.GetValor(k, j);
}
```

Operación matemática

Multiplica elementos correspondientes y acumula resultados y guarda el valor calculado dentro de la matriz resultado.

```
        resultado.SetValor(i, j, suma);
    }
}

return resultado;
}
```

2. Formulario Form1

El formulario es la interfaz gráfica donde interactúa el usuario. Almacenan los objetos de las matrices creadas.

```
3 referencias
public partial class Form1 : Form
{
    Matriz matrizA;
    Matriz matrizB;
}
```

Evento btnCrear_Click

Este evento se ejecuta al presionar el botón “Crear”. Se declaran variables para almacenar dimensiones.

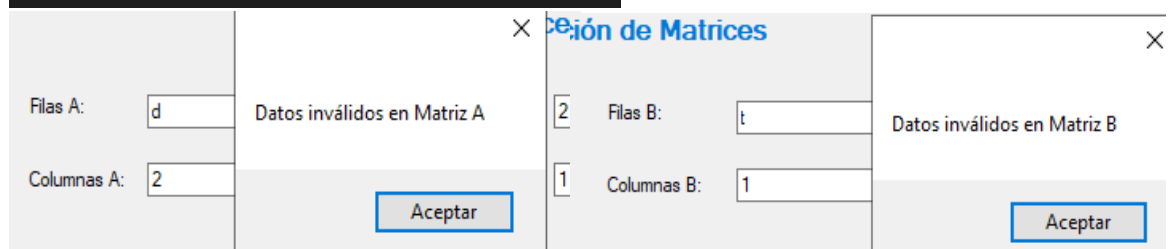
```
1 referencia
private void btnCrear_Click(object sender, EventArgs e)
{
    int filasA, columnasA;
    int filasB, columnasB;
}
```

Validación de datos

Convierte texto a número entero de forma segura.

```
// VALIDAR MATRIZ A
if (!int.TryParse(txtFilasA.Text, out filasA) ||
    !int.TryParse(txtColumnasA.Text, out columnasA))
{
    MessageBox.Show("Datos inválidos en Matriz A");
    return;
}

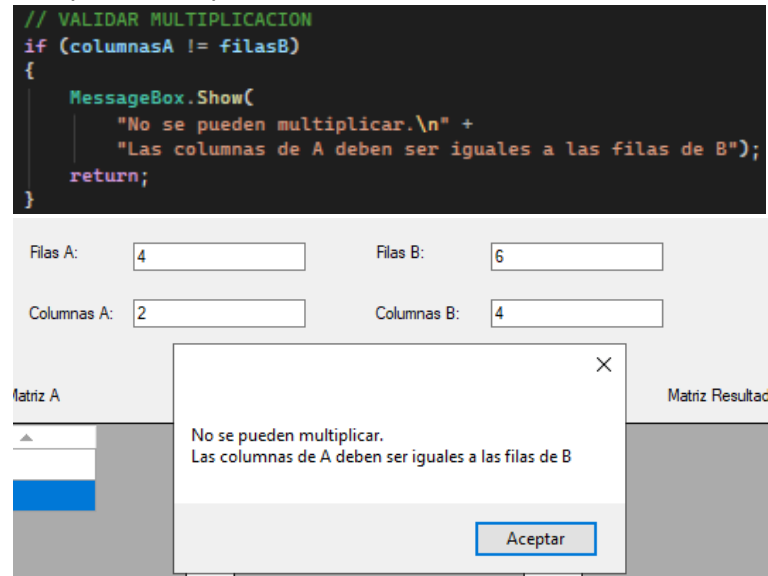
// VALIDAR MATRIZ B
if (!int.TryParse(txtFilasB.Text, out filasB) ||
    !int.TryParse(txtColumnasB.Text, out columnasB))
{
    MessageBox.Show("Datos inválidos en Matriz B");
    return;
}
```



The screenshot shows a dialog box titled "Creación de Matrices" with two sections for matrix creation. The left section for "Matriz A" has input fields for "Filas A:" (containing 'd') and "Columnas A:" (containing '2'). Below these fields is a message box that says "Datos inválidos en Matriz A" with an "Aceptar" button. The right section for "Matriz B" has input fields for "Filas B:" (containing 't') and "Columnas B:" (containing '1'). Below these fields is a message box that says "Datos inválidos en Matriz B" with an "Aceptar" button. The dialog box has a close button (X) in the top right corner.

Validación matemática

Comprueba compatibilidad entre matrices.



Configuración de DataGridView

Genera dinámicamente las celdas para ingresar datos.

```
// CREAR MATRIZ A
dgvA.Columns.Clear();
dgvA.Rows.Clear();

dgvA.ColumnCount = columnasA;
dgvA.RowCount = filasA;

// CREAR MATRIZ B
dgvB.Columns.Clear();
dgvB.Rows.Clear();

dgvB.ColumnCount = columnasB;
dgvB.RowCount = filasB;

// LIMPIAR RESULTADO
dgvR.Columns.Clear();
dgvR.Rows.Clear();

dgvR.AllowUserToAddRows = false;
```

Evitar filas vacías

Impide que aparezcan filas adicionales innecesarias.

```
// EVITAR FILAS VACIAS
dgvA.AllowUserToAddRows = false;

dgvB.AllowUserToAddRows = false;

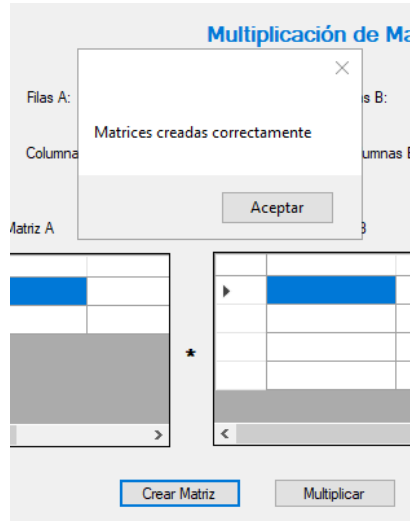
dgvR.AllowUserToAddRows = false;
```

Creación de objetos

Instancia las matrices usando el constructor.

```
// CREAR OBJETOS
matrizA = new Matriz(filasA, columnasA);
matrizB = new Matriz(filasB, columnasB);

MessageBox.Show("Matrices creadas correctamente");
}
```

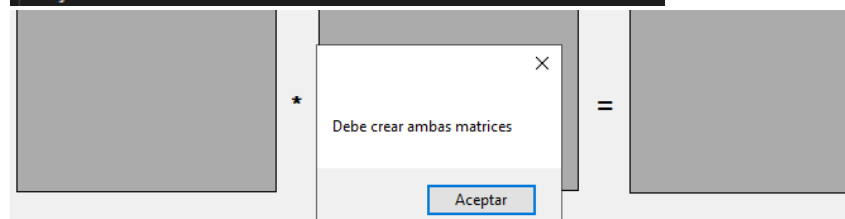


Evento btnMultiplicar_Click

Este evento ejecuta la multiplicación.

Verifica que las matrices hayan sido creadas.

```
1 referencia
private void btnMultiplicar_Click(object sender, EventArgs e)
{
    if (matrizA == null || matrizB == null)
    {
        MessageBox.Show("Debe crear ambas matrices");
        return;
    }
}
```



Lectura de datos del DataGridView

Convierte el contenido de las celdas a enteros y guarda los datos ingresados por el usuario.

```
try
{
    // LEER MATRIZ A
    for (int i = 0; i < matrizA.Filas; i++)
    {
        for (int j = 0; j < matrizA.Columnas; j++)
        {
            int valor = Convert.ToInt32(
                dgvA.Rows[i].Cells[j].Value);
        }
    }
}
```

```
// LEER MATRIZ B
for (int i = 0; i < matrizB.Filas; i++)
{
    for (int j = 0; j < matrizB.Columnas; j++)
    {
        int valor = Convert.ToInt32(
            dgvB.Rows[i].Cells[j].Value);

        matrizA.SetValor(i, j, valor);
    }

    matrizB.SetValor(i, j, valor);
}
```

Multiplicación

Llama al método estático de multiplicación.

```
// MULTIPLICAR
Matriz resultado = Matriz.Multiplicar(matrizA, matrizB);
```

Muestra el resultado final en pantalla.

```
// MOSTRAR RESULTADO
dgvR.ColumnCount = resultado.Columnas;
dgvR.RowCount = resultado.Filas;

dgvR.AllowUserToAddRows = false;

for (int i = 0; i < resultado.Filas; i++)
{
    for (int j = 0; j < resultado.Columnas; j++)
    {
        dgvR.Rows[i].Cells[j].Value =
            resultado.GetValor(i, j);
    }
}
```

Matriz A			Matriz B			Matriz Resultado					
	2	3		2	7	▶	11	41			
▶	1	5		3	6		12	39			
			▶	4	3		11	27			
				1	2		4	13			
<			>	<			>	<			>

Manejo de errores

Captura errores y evita que el programa se cierre inesperadamente.

```
catch
{
    MessageBox.Show("Complete todas las celdas");
}
```



Evento btnBorrar_Click

Este botón reinicia completamente el formulario.

Elimina datos ingresados y toda la información visual. Libera referencias de memoria y reinicia las matrices.

```
1 referencia
private void btnBorrar_Click(object sender, EventArgs e)
{
    txtFilasA.Clear();
    txtColumnasA.Clear();

    txtFilasB.Clear();
    txtColumnasB.Clear();

    dgvA.Columns.Clear();
    dgvA.Rows.Clear();

    dgvB.Columns.Clear();
    dgvB.Rows.Clear();

    dgvR.Columns.Clear();
    dgvR.Rows.Clear();

    matrizA = null;
    matrizB = null;
}
```

Programa: Áreas y perímetros

1. Diseño del Programa

¿QUÉ DESEAS CALCULAR?

Dato1

Dato2

Dato3

Dato4

Área: 0

Perímetro: 0

Calcular

Limpiar



2. Tabla de Controles

Control	Propiedad	Valor
Label1	Text	¿QUÉ DESEAS CALCULAR?
	Font	bold type
Label2	Text	Dato1
Label3	Text	Dato2
Label4	Text	Dato3
Label5	Text	Dato4
Label6	Name	lblArea
	Text	Área: 0
Label7	Name	lblPerimetro
	Text	Perímetro: 0
Button1	Name	btnCalcular
	Text	Calcular
Button2	Name	btnLimpiar
	Text	Limpiar
textBox1	Name	txtDato1
textBox2	Name	txtDato2
textBox3	Name	txtDato3
textBox4	Name	txtDato4
dataGridView1	Name	cboFigura

3. Código del Programa

El proyecto está dividido en:

- Clase base Figura
- Clases derivadas:
 - Cuadrado
 - Rectangulo
 - Triangulo
 - Circulo

1. Formulario Form1

Clase Base Figura

La clase Figura representa una clase general de la cual heredarán las demás figuras geométricas. Define una clase base común para todas las figuras.

```
{
    9 referencias
    public class Figura
    {
```

Métodos Virtuales

Método CalcularArea

Define un método general para calcular áreas. Se utiliza virtual para permitir que las clases hijas modifiquen este comportamiento mediante override.

```
// Métodos virtuales (Polimorfismo)
5 referencias
public virtual double CalcularArea()
{
    return 0;
}
```

Método CalcularPerimetro

Define un método general para calcular perímetros. Gracias al polimorfismo, una variable de tipo Figura puede almacenar objetos de diferentes clases derivadas.

```
5 referencias
public virtual double CalcularPerimetro()
{
    return 0;
}
```

Clase Circulo

La clase Circulo hereda de Figura.

Indica que Circulo hereda características de Figura.

```
2 referencias
public class Circulo : Figura
{
}
```

Encapsulamiento

Almacena el radio del círculo. El atributo es privado para proteger los datos.

```
private double radio;
```

Constructor

Inicializa el radio del círculo.

```
1 referencia
public Circulo(double radio)
{
    this.radio = radio;
}
```

Sobrescritura del método CalcularArea

Calcula el área del círculo utilizando la fórmula matemática:

```
2 referencias
public override double CalcularArea()
{
    return Math.PI * radio * radio;
}
```

Sobrescritura del método CalcularPerimetro

Calcula el perímetro o circunferencia:

```
2 referencias
public override double CalcularPerimetro()
{
    return 2 * Math.PI * radio;
}
```

Clase Rectangulo

La clase Rectangulo hereda de Figura.

Almacenan la base y altura del rectángulo.

```
2 referencias
public class Rectangulo : Figura
{
    private double baseR;
    private double altura;
}
```

Constructor

Inicializa las dimensiones del rectángulo.

```
1 referencia
public Rectangulo(double baseR, double altura)
{
    this.baseR = baseR;
    this.altura = altura;
}
```

Método CalcularArea

```
2 referencias
public override double CalcularArea()
{
    return baseR * altura;
}
```

Método CalcularPerimetro

```
2 referencias
public override double CalcularPerimetro()
{
    return 2 * (baseR + altura);
}
```

Clase Triangulo

La clase Triangulo también hereda de Figura.

Almacenan los datos necesarios para calcular área y perímetro.

```
2 referencias
public class Triangulo : Figura
{
    private double baseT;
    private double altura;
    private double lado1;
    private double lado2;
}
```

Constructor

Recibe los valores necesarios para inicializar el triángulo.

```
1 referencia
public Triangulo(double baseT, double altura,
                 double lado1, double lado2)
{
    this.baseT = baseT;
    this.altura = altura;
    this.lado1 = lado1;
    this.lado2 = lado2;
}
```

Método CalcularArea

```
2 referencias
public override double CalcularArea()
{
    return (baseT * altura) / 2;
}
```

Método CalcularPerimetro

```
2 referencias
public override double CalcularPerimetro()
{
    return baseT + lado1 + lado2;
}
```

Clase Cuadrado

La clase Cuadrado hereda de Figura.

Guarda el tamaño del lado del cuadrado.

```
2 referencias
public class Cuadrado : Figura
{
    // Encapsulamiento
    private double lado;
}
```

Constructor

Inicializa el lado del cuadrado.

```
// Constructor
1 referencia
public Cuadrado(double lado)
{
    this.lado = lado;
}
```

Método CalcularArea

```
// Sobrescritura
2 referencias
public override double CalcularArea()
{
    return lado * lado;
}
```


Método CalcularPerimetro

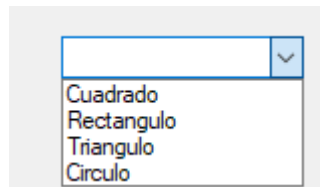
```
2 referencias
public override double CalcularPerimetro()
{
    return 4 * lado;
}
```

Formulario Form1

Agregar elementos al ComboBox

Carga las figuras disponibles dentro del ComboBox.

```
1 referencia
private void Form1_Load(object sender, EventArgs e)
{
    cboFigura.Items.Add("Cuadrado");
    cboFigura.Items.Add("Rectangulo");
    cboFigura.Items.Add("Triangulo");
    cboFigura.Items.Add("Circulo");
}
```



Evento cboFigura_SelectedIndexChanged

Este evento ocurre cuando el usuario cambia la figura seleccionada. Hace visibles los controles necesarios.

```
1 referencia
private void cboFigura_SelectedIndexChanged(object sender, EventArgs e)
{
    lblDato1.Visible = true;
    lblDato2.Visible = true;
    lblDato3.Visible = true;
    lblDato4.Visible = true;

    txtDato1.Visible = true;
    txtDato2.Visible = true;
    txtDato3.Visible = true;
    txtDato4.Visible = true;
}
```

Uso del switch

Evalúa qué figura fue seleccionada.

```
switch (cboFigura.Text)
{
```

Caso Cuadrado

Solo necesita un dato: lado. Por ello se ocultan controles innecesarios.

```
case "Cuadrado":  
  
    lblDato1.Text = "Lado";  
  
    lblDato2.Visible = false;  
    lblDato3.Visible = false;  
    lblDato4.Visible = false;  
  
    txtDato2.Visible = false;  
    txtDato3.Visible = false;  
    txtDato4.Visible = false;  
  
    break;
```

¿QUE DESEAS CALCULAR?

Lado Cuadrado ▾

Caso Rectangulo

Solicita base y altura.

```
case "Rectangulo":  
  
    lblDato1.Text = "Base";  
    lblDato2.Text = "Altura";  
  
    lblDato3.Visible = false;  
    lblDato4.Visible = false;  
  
    txtDato3.Visible = false;  
    txtDato4.Visible = false;  
  
    break;
```

Base Rectangulo ▾

Altura

Caso Triangulo

Solicita base, altura, lado 1 y lado 2

```
case "Triangulo":  
  
    lblDato1.Text = "Base";  
    lblDato2.Text = "Altura";  
    lblDato3.Text = "Lado 1";  
    lblDato4.Text = "Lado 2";  
  
    break;
```

Base Triangulo ▾

Altura

Lado 1

Lado 2

Área: 0
Perímetro: 0

Caso Circulo

Solicita únicamente el radio.

```
case "Circulo":  
  
    lblDato1.Text = "Radio";  
  
    lblDato2.Visible = false;  
    lblDato3.Visible = false;  
    lblDato4.Visible = false;  
  
    txtDato2.Visible = false;  
    txtDato3.Visible = false;  
    txtDato4.Visible = false;  
  
    break;
```



Evento btnCalcular_Click

Este evento realiza los cálculos.

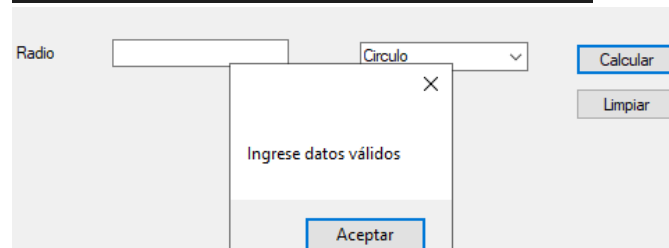
La variable puede almacenar cualquier figura derivada.

```
1 referencia  
private void btnCalcular_Click(object sender, EventArgs e)  
{  
    Figura figura = null;
```

Uso del try-catch

Captura errores si el usuario escribe datos incorrectos.

```
catch  
{  
    MessageBox.Show("Ingrese datos válidos");  
}
```



Creación dinámica de objetos

Se crea el objeto dependiendo de la figura seleccionada.

Cuadrado

```
switch (cboFigura.Text)  
{  
    case "Cuadrado":  
  
        double lado = Convert.ToDouble(txtDato1.Text);  
  
        figura = new Cuadrado(lado);  
  
        break;
```



Lado Cuadrado

Área: 9.00
Perímetro: 12.00

Rectángulo

```
case "Rectangulo":  
  
    double baseR = Convert.ToDouble(txtDato1.Text);  
    double alturaR = Convert.ToDouble(txtDato2.Text);  
  
    figura = new Rectangulo(baseR, alturaR);  
  
    break;
```

Base Rectangulo

Altura

Área: 15.00
Perímetro: 16.00

Triángulo

```
case "Triangulo":  
  
    double baseT = Convert.ToDouble(txtDato1.Text);  
    double alturaT = Convert.ToDouble(txtDato2.Text);  
    double lado1 = Convert.ToDouble(txtDato3.Text);  
    double lado2 = Convert.ToDouble(txtDato4.Text);  
  
    figura = new Triangulo(baseT, alturaT, lado1, lado2);  
  
    break;
```

Base Triangulo

Altura

Lado 1

Lado 2

Área: 7.50
Perímetro: 9.00

Círculo

```
case "Circulo":  
  
    double radio = Convert.ToDouble(txtDato1.Text);  
  
    figura = new Circulo(radio);  
  
    break;
```



Radio 3 Circulo Calcular Limpiar

Área: 28.27
Perímetro: 18.85

Cálculo del Área

Aunque la variable es de tipo Figura, se ejecuta el método correspondiente a cada clase. Esto ocurre gracias al polimorfismo.

```
lblArea.Text =  
    "Área: " + figura.CalcularArea().ToString("0.00");
```

Cálculo del Perímetro

Calcula el perímetro específico de cada figura.

```
lblPerimetro.Text =  
    "Perímetro: " + figura.CalcularPerimetro().ToString("0.00");  
}
```

Evento btnLimpiar_Click

Este botón reinicia el formulario.

Elimina los datos ingresados y restablece los textos originales.

```
1 referencia  
private void btnLimpiar_Click(object sender, EventArgs e)  
{  
    txtDato1.Clear();  
    txtDato2.Clear();  
    txtDato3.Clear();  
    txtDato4.Clear();  
  
    lblArea.Text = "Área:";  
    lblPerimetro.Text = "Perímetro:";  
}
```



IV. ANALISIS E INTERPRETACION DE RESULTADOS

¿Qué indican los resultados?

- El sistema fue capaz de multiplicar correctamente matrices.
- Se verificó de manera correcta la compatibilidad de las filas con las columnas.
- Los resultados obtenidos se alinean con los que se podían llegar a esperar al realizar dichas operaciones matemáticas.
- Gracias a la interfaz gráfica fue posible introducir y apreciar matrices de forma dinámica.
- El sistema fue capaz de calcular correctamente el área y perímetro para las diferentes figuras geométricas.
- El uso de herencia y polimorfismo permitió la reutilización de código.
- La interfaz gráfica reaccionaba de forma dinámica a la figura seleccionada.
- Los resultados eran mostrados con el grado de aproximación esperado, en términos decimales.

¿Que se ha encontrado?

- La validación previa permite evitar errores en la multiplicación.
- El encapsulamiento permite asegurar los datos.
- Utilizar DataGridView permite manipular matrices.
- Utilizar polimorfismo permite trabajar con parecidas figuras geométricas.
- Utilizar herencia permite reducir la duplicidad del código.
- Utilizar virtual y override permite construir un código más flexible para el sistema.

CONCLUSIONES

- Se lograron aplicar correctamente los principios de Programación Orientada a Objetos en C#.
- El Programa permitió entender la gestión de matrices y operaciones matemáticas haciendo uso de clases y objetos.
- El Programa “Áreas y perímetros” hizo posible realizar la herencia y el polimorfismo haciendo uso de figuras geométricas.
- Windows Forms permitió hacer interface gráficas dinámicas e interactivas.
- Las validaciones implementadas mejoraron la estabilidad y el funcionamiento del sistema.



RECOMENDACIONES

- Implementar mensajes de error más específicos.
- Agregar más figuras geométricas al sistema.
- Permitir guardar resultados en archivos.
- Mejorar el diseño visual de la interfaz para una experiencia más amigable.

WEBGRAFIA

BillWagner. (s. f.). Operadores y expresiones: enumerar todos los operadores y expresiones - C# reference. Microsoft Learn. Recuperado de <https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/operators/>

Arreglos en C#. (2014). C# Paso a Paso. Recuperado de <https://csharp-facilito.blogspot.com/2013/07/arreglos-en-c-sharp.html>

Nivardo. (2024). POO en C#. Oregoom.com. Recuperado de <https://oregoom.com/c-sharp/poo/>